Roteiro do vídeo — Tech Challenge Fase 2

ToggleMaster: de monolito a microsserviços no Amazon EKS

Versão revisada: 30/10/2026

Duração estimada: 18 a 19 minutos

Formato: duas partes, com troca de apresentador aos 9 minutos

> *Este roteiro considera a infraestrutura final totalmente implantada e funcional. Antes da gravação, os nomes e estados exibidos no Console e no terminal serão confrontados com o ambiente configurado.*

Divisão do vídeo

Parte	Conteúdo	Tempo
Parte 1	Abertura, arquitetura, Docker, ambiente local, infraestrutura AWS e persistência	≈ 9 min
Parte 2	Kubernetes, segredos, Ingress, teste ponta a ponta, HPA, DynamoDB e encerramento	≈ 10 min
Total	Limite do enunciado: 20 minutos	≈ 19 min

Trechos marcados como [opcional] podem ser removidos na edição se o vídeo ultrapassar o limite.

Conferir antes de gravar

â€¢ Os nove containers locais estão saudáveis.

â€¢ O contexto atual do kubectl aponta para o EKS correto.

â€¢ Os cinco Deployments estão disponíveis e os três Jobs SQL estão concluídos.

â€¢ Os quatro ExternalSecret estão sincronizados.

â€¢ O Load Balancer responde nas rotas /auth, /flags, /rules e /evaluate.

â€¢ Os HPAs de evaluation e analytics recebem métricas do Metrics Server.

â€¢ A carga para demonstrar o scale-out foi ensaiada.

â€¢ A tabela ToggleMasterAnalytics recebe os eventos do teste.

â€¢ Nenhuma tela mostra .env, AWS Details, senhas, tokens, valores do Secrets Manager, MASTER_KEY, API key ou credenciais temporárias.

Parte 1 — Arquitetura, ambiente local e AWS

1A — Abertura e arquitetura — 2 min

Deixar pronto

â€¢ Slide de abertura.

â€¢ Diagrama docs/arquitetura_aws_togglemaster.png.

Tempo	Na tela	Fala
0:00	Slide de título	Olá. Somos o grupo formado por Afonso Victoriano Franco, Cesar Augusto Ferreira Alves, Jeferson Lima Papalino, Jonatas Ariel Nunes e Willian Felix Corrêa. Vamos apresentar a Fase 2 do Tech

Tempo	Na tela	Fala
		Challenge: a evolução do ToggleMaster de um monolito para uma arquitetura de microsserviços executada no Amazon EKS.
0:30	Diagrama geral	Na Fase 1, o ToggleMaster era uma aplicação monolítica executada em EC2 e conectada a um PostgreSQL no RDS. Nesta fase, a solução foi dividida em cinco microsserviços com responsabilidades independentes.
0:55	Serviços no diagrama	O auth-service, em Go, cria e valida chaves de API. O flag-service, em Python, mantém as definições das feature flags. O targeting-service, também em Python, mantém as regras de segmentação. O evaluation-service, em Go, consulta flags e regras, aplica a decisão para cada usuário e usa Redis como cache. Por fim, o analytics-service consome os eventos e grava os dados de uso.
1:35	Fluxo Evaluation → SQS → Analytics → DynamoDB	O caminho de analytics é assíncrono. O evaluation publica cada resultado na fila SQS e devolve a resposta ao cliente sem aguardar a persistência analítica. O analytics consome a fila no próprio ritmo e grava no DynamoDB. Assim, uma lentidão no processamento analítico não bloqueia a avaliação da flag.

1B — Conteinerização e ambiente local — 2 min 30 s

Deixar pronto

â€¢ VS Code com um Dockerfile Go e um Dockerfile Python abertos.

â€¢ Terminal na raiz do projeto com o Docker Compose em execução.

Tempo	Na tela	Fala
2:00	Dockerfiles de auth-service e flag-service	Os cinco serviços têm Dockerfiles multi-stage. Nos serviços Go, o código é compilado no primeiro estágio e apenas o binário é levado para uma imagem Alpine. Nos serviços Python, as dependências são preparadas no estágio de build e copiadas para a imagem final. Todos os containers de aplicação executam com usuário sem privilégio de root.
2:40	docker images	A separação dos estágios evita levar compiladores e arquivos desnecessários para a imagem de execução. Aqui também podemos comparar os tamanhos reais das imagens geradas, sem citar valores fixos antes da coleta.
3:00	docker compose ps	Antes da implantação em nuvem, validamos a solução localmente com nove containers: cinco aplicações, dois PostgreSQL, um Redis e um LocalStack. O LocalStack emula as APIs de SQS e DynamoDB usadas no fluxo assíncrono.
3:35	Health checks	Os cinco serviços respondem aos health checks nas portas 8001 a 8005. Isso confirma autenticação, flags, targeting, evaluation e analytics antes de avançarmos para a AWS.
4:00	Relatório ou histórico de correções	[opcional] Durante a implementação, também corrigimos problemas de compilação, imports Go sem uso, compatibilidade do cliente Redis com o ElastiCache e diferenças de ambiente entre Windows, containers e AWS.

Comandos

```
docker images | grep -E "auth-service|flag-service|targeting-service|evaluation-service|analytics-service"
docker compose ps
for p in 8001 8002 8003 8004 8005; do
  echo -n "$p: "
  curl -s "localhost:$p/health"
  echo
done
```

1C — Infraestrutura AWS e persistência — 4 min 30 s

Deixar pronto

â€¢ Console AWS nas páginas do EKS, ECR, RDS, ElastiCache, DynamoDB, SQS e Security Groups.

â€¢ Região us-east-1 selecionada.

Tempo	Na tela	Fala
4:30	EKS — cluster	A infraestrutura foi criada no AWS Academy, na região us-east-1. O núcleo é o cluster Amazon EKS do ToggleMaster, responsável pelo plano de controle do Kubernetes.
4:55	EKS — Compute	Os workloads são executados por um Managed Node Group com capacidade mínima de um nó, desejada de dois e máxima de quatro. Os nós usam instâncias t3.medium e a LabRole, respeitando as limitações do ambiente acadêmico.
5:25	ECR	As cinco imagens ficam em repositórios privados no Amazon ECR, um para cada microsserviço. A imagem atual do evaluation contém a correção do cliente Redis para operar com TLS e cluster mode no ElastiCache.
5:55	RDS	Para dados relacionais, criamos três instâncias PostgreSQL privadas no RDS: uma para autenticação, uma para flags e outra para targeting. Essa separação segue o padrão database-per-service e impede que um serviço acesse diretamente as tabelas de outro domínio.
6:40	ElastiCache	O evaluation usa Redis no ElastiCache, com TLS e token de autenticação obrigatório, para armazenar por 30 segundos os dados combinados da flag e da regra. Em um cache hit, ele evita novas chamadas aos serviços de flags e targeting, reduzindo a latência do caminho crítico.
7:15	DynamoDB	Os eventos analíticos são persistidos na tabela ToggleMasterAnalytics. O modelo NoSQL se adapta ao formato dos eventos e suporta volume variável de escrita sem acoplar o analytics aos bancos transacionais.
7:45	SQS	Entre evaluation e analytics está a fila Standard tgmqueue. Ela desacopla os serviços e absorve variações de carga. Como filas Standard podem entregar uma mensagem mais de uma vez, idempotência e DLQ são evoluções recomendadas para produção.
8:15	Security Groups	Os bancos e o cache não são públicos. Os Security Groups permitem acesso ao RDS na porta 5432 e ao Redis na porta 6379 somente a partir da origem autorizada do EKS.
8:40	Diagrama	Com as imagens e os serviços gerenciados prontos, vamos mostrar como os microsserviços são executados e expostos dentro do cluster.

Parte 2 — Kubernetes, escalabilidade e encerramento

2A — Manifestos, segredos, Ingress e fluxo funcional — 5 min

Deixar pronto

â€¢ Terminal com os pods em estado Running.

â€¢ MASTER_KEY carregada em variável de ambiente, sem aparecer no terminal.

â€¢ Variável LB preenchida com o endereço atual do Load Balancer.

â€¢ Uma flag de demonstração que ainda não exista.

Tempo	Na tela	Fala
9:00	Pasta k8s-aws e kustomization.yaml	Os manifestos estão na pasta k8s-aws deste repositório, organizados por serviço. O Kustomize reúne os recursos e permite aplicar o conjunto de forma consistente.
9:25	kubectl get ns	Cada microsserviço possui seu próprio namespace: auth-service, flag-service, targeting-service, evaluation-service e analytics-service. Os serviços se comunicam pelo DNS interno do cluster, como flag-service.flag-service.svc.cluster.local.
9:50	Deployment do flag-service	Os Deployments definem startup, readiness e liveness probes; requests e limits de CPU e memória; e controles de segurança. Os containers executam sem root, sem elevação de privilégio, com as capabilities removidas e o sistema de arquivos raiz somente para leitura.
10:40	ExternalSecret, mostrando apenas nomes e status	As credenciais não ficam gravadas nos YAMLs. O AWS Secrets Manager guarda URLs de banco, chaves internas e o token do Redis. O External Secrets Operator sincroniza esses valores para Secrets do Kubernetes. No AWS Academy, os componentes acessam a AWS pela LabRole associada aos nós, sem chaves estáticas nos manifestos.
11:25	kubectl get jobs -A	Três Jobs executam os scripts SQL de inicialização nos bancos de auth, flag e targeting. Isso torna a criação das tabelas reproduzível e substitui uma etapa manual.

Tempo	Na tela	Fala
11:50	kubectl get pods,svc -A	Os cinco Deployments estão disponíveis. Cada serviço possui um ClusterIP, acessível somente dentro do cluster.
12:15	kubectl get ingress -A	Para a entrada externa, as rotas /auth, /flags, /rules e /evaluate passam pelo mesmo Load Balancer e pelo Ingress Controller. O analytics não possui rota pública porque funciona como worker interno.
12:45	Ingress e slide de evolução	O ambiente acadêmico usa ingress-nginx, que continua funcionando, mas foi aposentado pelo projeto Kubernetes em 24 de março de 2026 e não recebe novas correções. Em produção, a migração deve ser planejada para Gateway API ou outro controller mantido.
13:10	Chamadas pelo Load Balancer	Agora executamos o fluxo externo: criamos uma API key, cadastramos uma flag e uma regra de targeting de 100%, e consultamos o evaluation. A resposta esperada é result: true.

Comandos de inspeção

```
kubectl config current-context
kubectl get ns
kubectl get externalsecret -A
kubectl get jobs -A
kubectl get pods,svc -A | grep -E "auth|flag|targeting|evaluation|analytics"
kubectl get ingress -A
```

Comandos do fluxo funcional

Preparar e testar antes da gravação. Os comandos armazenam as chaves em variáveis e não as imprimem.

```
LB=$(kubectl get ingress -A -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}')
FLAG="enable-new-dashboard"

KEY=$(curl -s -X POST "http://$LB/auth/admin/keys" \
  -H "Authorization: Bearer $MASTER_KEY" \
  -H "Content-Type: application/json" \
  -d '{"name":"demo-video"}' | jq -r '.key')

curl -s -X POST "http://$LB/flags" \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d "{\"name\":\"$FLAG\",\"description\":\"Demonstração do vídeo\",\"is_enabled\":true}"

curl -s -X POST "http://$LB/rules" \
  -H "Authorization: Bearer $KEY" \
  -H "Content-Type: application/json" \
  -d "{\"flag_name\":\"$FLAG\",\"is_enabled\":true,\"rules\":{\"type\":\"PERCENTAGE\",\"value\":100}}"

curl -s "http://$LB/evaluate?user_id=user-123&flag_name=$FLAG"
```

2B — Escalabilidade, DynamoDB e encerramento — 5 min

Deixar pronto

â€¢ Três terminais: geração de carga, acompanhamento do HPA e acompanhamento dos pods.

â€¢ Flag enable-new-dashboard ativa.

â€¢ Load Generator ensaiado para saber quantos pods são necessários para ultrapassar o alvo.

Tempo	Na tela	Fala
14:00	kubectl top nodes e kubectl top pods -A	O Metrics Server coleta o consumo de CPU e memória dos nós e pods. Essas métricas alimentam o Horizontal Pod Autoscaler.
14:25	kubectl get hpa -A	Configuramos HPAs para evaluation e analytics. No ambiente AWS, cada um trabalha entre uma e cinco réplicas, com alvo médio de CPU de 30% do request definido no Deployment.
14:55	Terminal de carga	Vamos gerar chamadas contínuas para o evaluation, simulando vários clientes consultando a mesma flag.

Tempo	Na tela	Fala
15:20	HPA e pods em modo watch	A utilização de CPU ultrapassa o alvo. O HPA aumenta o número desejado de réplicas e novos pods do evaluation são criados.
16:05	Novas réplicas Running	O evaluation passou de uma para as réplicas mostradas na tela sem intervenção manual. O Service continua oferecendo um único endereço e distribui as requisições entre os pods disponíveis.
16:35	Carga encerrada	Quando interrompemos a carga, o HPA respeita a janela de estabilização de 60 segundos e reduz as réplicas. Isso libera capacidade dentro do cluster. A redução do custo dos nós exigiria também autoscaling da infraestrutura; o HPA sozinho não desliga instâncias EC2.
17:15	DynamoDB — itens da tabela	Cada avaliação publica um evento no SQS. O analytics consome a mensagem e grava um item no DynamoDB. Estes registros comprovam o fluxo ponta a ponta entre a API, a fila, o worker e a persistência analítica.
17:50	Slide de lições aprendidas	As principais lições foram validar localmente antes da nuvem, manter responsabilidades e bancos separados, proteger segredos fora dos manifestos, observar o contexto do kubectl e registrar evidências antes, durante e depois dos testes.
18:25	Slide de próximos passos	[opcional] Como próximos passos, propomos CI/CD para build e deploy, HTTPS, observabilidade, EKS Pod Identity ou IRSA, DLQ e idempotência no fluxo assíncrono, Gateway API e autoscaling também para os nós.
18:50	Slide final	Obrigado. Este foi o ToggleMaster na Fase 2: uma plataforma de microsserviços conteinerizada, implantada no EKS e integrada a serviços gerenciados da AWS.

Comandos de métricas e acompanhamento

```
kubectl top nodes
kubectl top pods -A
kubectl get hpa -A
kubectl get hpa -n evaluation -w
kubectl get pods -n evaluation -w
```

Geração de carga

```
for i in 1 2 3; do
  kubectl run "carga-$i" \
    --image=busybox:1.36 \
    --restart=Never \
    -- /bin/sh -c \
    'while true; do wget -q -O- "http://evaluation-
service.evaluation.svc.cluster.local:8004/evaluate?user_id=user-123&flag_name=enable-new-dashboard"
>/dev/null; done'
done
```

Encerrar a carga

```
kubectl delete pod carga-1 carga-2 carga-3 --ignore-not-found
```

Prova no DynamoDB

```
aws dynamodb scan \
  --region us-east-1 \
  --table-name ToggleMasterAnalytics \
  --max-items 5 \
  --output table
```

Dicas de gravação

â€¢ Gravar cada parte separadamente e unir na edição.

â€¢ Aumentar a fonte do terminal e ocultar notificações.

â€¢ Deixar comandos testados em um arquivo local para copiar e colar.

â€¢ Não abrir arquivos de credenciais ou mostrar a saída de env, set, Secrets ou AWS Details.

â€¢ Cortar tempos de espera de criação de pods e estabilização do HPA, mantendo a sequência de evidências.

â€¢ Mostrar a hora ou um marcador visual nos estados antes, durante e depois da carga.

â€¢ Confirmar que o vídeo final não ultrapassa 20 minutos.

Ajustes feitos em relação ao PDF original

â€¢ Corrigido “DynamoDB Local” para LocalStack, que é o container usado pelo Compose para emular DynamoDB e SQS.

â€¢ Corrigida a afirmação de que os manifestos ficam em repositório separado; eles estão em k8s-aws no mesmo repositório.

â€¢ Especificado que o Redis armazena os dados combinados de flag e regra por 30 segundos.

â€¢ Incluída a criação da regra de targeting no teste ponta a ponta.

â€¢ Corrigido e completado o comando de geração de carga, que estava truncado no PDF.

â€¢ Corrigida a explicação de custo: reduzir pods com HPA não reduz sozinho o custo dos nós EC2.

â€¢ Removidos tamanhos fixos de imagens sem coleta atual reproduzível.

â€¢ Confirmados três RDS e os parâmetros user_id e flag_name de /evaluate.

â€¢ Atualizada a informação sobre o ingress-nginx: aposentado em 24/03/2026, embora instalações existentes continuem funcionando.